

Maxims of the Quackulator

Playing The Quacks of Quedlinburg to win, not to score

Corey D. — Quackulator project, v3 (Set 1 ingredients)

September 2026

Abstract

The Quacks of Quedlinburg is a nine-round push-your-luck bag-building game. Each round a player draws chips blind from a bag, advancing a marker around a pot; white "cherry bomb" chips accumulate, and when their total exceeds 7 the pot explodes. The player may stop before any draw. The scoring space reached pays coins (spent on new chips), victory points, and sometimes a ruby. The Quackulator is a calculator that, given the true contents of the player's bag, the state of the pot and the score of the best opponent, recommends the action that maximises the player's **probability of winning the game**. This paper states the principles behind those recommendations as a set of *maxims*: each is a plain-language rule, followed by the mathematics that justifies it and, where useful, a measured number from the engine. Three generations of the engine are compared on a simulated four-player table: a hand-tuned expected-score policy (v1), a self-play policy that maximises expected score (v2), and the present policy, which maximises win probability (v3). The v3 policy wins 34 % of four-player games against three copies of v2, where an equal player would win 25 %. The in-round solver is exact; the learned layer and its remaining assumptions are stated so that a sceptical reader can test them against their own table.

1. The problem, stated precisely

A round begins with a bag B , a multiset of chips. Each chip has a colour c and a value $v \in \{1, 2, 3, 4\}$. White chips (W) are the bombs. The pot is a track of 54 spaces indexed 0...53; space s pays coins(s), VP(s) and ruby(s) $\in \{0, 1\}$ as printed on the board (Table 1). The droplet sits on space d ; the first chip is placed v spaces after it, each later chip v spaces after the previous one. The **scoring space** is the space *after* the last chip. If the sum of white values placed exceeds the limit $L = 7$ the pot explodes; the exploding chip still counts for position, but the player then keeps *either* the coins *or* the VP of the scoring space, not both, and cannot roll the bonus die.

The state during a brew is

| $S =$ (bag remaining, position, white sum, last-white value, green flags, flask available).

The player's decision at every state is binary — **stop** or **draw** — with three side-decisions that arise on particular chips: whether to use the flask on a white chip just drawn, whether a mandrake (yellow) returns the preceding white, and which of the chips revealed by a crow skull (blue) to place. After the round come the evaluation choices: VP-or-coins if exploded, what to buy with the coins, and how to spend rubies.

Between rounds the game couples the players through three channels: **rat tails** (a player behind the leader on the score track starts the next brew further along the pot, one space per rat-tail mark between the two scores), the **bonus die** (only the furthest non-exploded pot rolls it), and **black chips**

(compared with the two neighbours' counts). These are the whole of the interaction; a player never acts on another player's pot.

The Quackulator's claim is that its recommendations maximise

$$P(\text{the player finishes with the highest score}),$$

taken over the randomness of every bag at the table. Sections 2 and 3 show that within a round this objective is served by an exact search; Section 4 shows how the value of a round's outcome is learned; Section 5 explains what changes, and what does not, when the objective is a win rather than a score.

Table 1. Pot track (space index → coins / VP, ★ = ruby). Coin values 15–33 appear twice; the last space pays 35 / 15.

idx	0–4	5★	6–8	9★	10–12	13★	14	15	16★	17	18	19	20★	21	22	23	24★	25	26
coins	0–4	5	6–8	9	10–12	13	14	15	15	16	16	17	17	18	18	19	19	20	20
VP	0	0	1	1	2	2	3	3	3	3	4	4	4	4	5	5	5	5	6

idx	27	28★	29	30★	31	32	33	34★	35	36★	37	38	39	40★
coins	21	21	22	22	23	23	24	24	25	25	26	26	27	27
VP	6	6	7	7	7	8	8	8	9	9	9	10	10	10

idx	41	42★	43	44	45	46★	47	48	49	50★	51	52★	53
coins	28	28	29	29	30	30	31	31	32	32	33	33	35
VP	11	11	11	12	12	12	12	13	13	13	14	14	15

2. Maxims of drawing

Maxim 1 — Draw when the expected value of drawing exceeds the value of stopping; there is no other rule.

Every "rule of thumb" for push-your-luck games (stop at five bombs, never draw above 30 %, and so on) is a compression of one comparison. Let $V(S)$ be the value of being at state S with the choice still open. Then

$$V(S) = \max(\text{Stop}(S), \text{Draw}(S))$$

$$\text{Draw}(S) = \sum \text{ over chip types } i \text{ of } (n_i / n) \cdot \text{Place}(S, i)$$

where n_i is the number of chips of type i still in the bag, n the total, and $\text{Place}(S, i)$ is the value after chip i is placed and its action resolved (recursively calling V unless the pot exploded). $\text{Stop}(S)$ is the terminal value of the current scoring space (Section 4). This is the Bellman equation of a finite Markov decision process: the bag is finite, every draw removes a chip, so the recursion terminates, and the maximum at each node is exactly the optimal policy. The advisor's DRAW/STOP is simply the sign of $\text{Draw}(S) - \text{Stop}(S)$.

The consequence people find surprising is that **the decision is not a function of the bomb count alone**. Two states with five bombs in the pot can have opposite answers depending on which whites remain in the bag and what the coloured chips would do. Nothing in this maxim depends on what the terminal value *measures*: whether $Stop(S)$ is a score or a win probability, the recursion and the optimality argument are the same. What changes with the objective is the shape of $Stop$, and Section 5 shows that this is where risk appetite enters.

Maxim 2 — A draw that cannot explode is always taken.

If every white left in the bag satisfies $white-sum + v \leq L$, then $Place(S, i) \geq Stop(S)$ for every i : each chip moves the marker forward, and $coins(s)$, $VP(s)$ are non-decreasing in s , so every terminal value that is monotone in the outcome (score and win probability both are) is non-decreasing too. Hence $Draw(S) \geq Stop(S)$, strictly whenever any chip has positive value. The advisor reports this case as "No bomb can blow you up on this draw". It is the one situation where the human rule and the mathematics coincide exactly.

Maxim 3 — An explosion is a purchase: early ones are cheap, and the price is what the coins would have become.

An explosion does not zero the round: the player keeps coins *or* VP. The engine values an exploded pot at scoring space s as

$$| \quad Boom(s) = \max(T(VP(s), \text{no coins}), T(0 \text{ VP}, \text{coins}(s))),$$

where T is the terminal value of Section 4, which already includes the best purchase the coins can make and the whole future of the bag that results. There is no exchange rate; a bust is priced by what it does to the rest of the game. Measured over 3 000 solo games (Appendix A), the v3 policy explodes in 62 % of first rounds and 54 % of second rounds, and takes the coins every time through round 5. From round 7 on it never takes coins. This is the same pattern the v2 policy discovered, and it is the largest single departure from human play: a deliberate first-round bust for ten coins is a normal opening.

Maxim 4 — The flask is a bomb eraser; use it when the position it saves is worth more than the two rubies it costs.

Drawing a white v that does not explode moves the state from S to S' . The flask offers a third branch: return the chip, restoring the bag and position of S but marking the flask used. The engine evaluates

$$| \quad Place(S, Wv) = \max(V(S'), V(S \text{ with flask spent}))$$

and the flask is recommended when the second term is larger. The comparison is "is being v bombs lighter worth the refill?", and the refill's worth is not a fixed number: it is the difference in the learned value between a bag state with a full flask and one without (Section 4), which is large in round 2 and nearly nothing in round 9. Empirically the answer is yes for a W3 drawn mid-round with a bag full of coloured chips, and no for a W1 drawn at the start.

Maxim 5 — A mandrake almost always returns the white before it.

The mandrake (yellow) may return the white chip placed directly before it; the yellow then takes that white's spot. The white is put back into the bag, so it can be drawn again — but the immediate effect is a

reduction of the bomb count by v at a cost of v spaces, with no ruby cost. Because the marginal space is worth little and the marginal bomb is worth far more once the count is high, the engine's `max(keep, return)` chooses "return" in nearly every state where the preceding chip was white. The advisor still shows both values, because the exception exists: a W1 before a Y4 late in a round with no bombs left to fear. (The learned policies buy almost no yellow — 0.04 per game — so the maxim is rarely exercised.)

Maxim 6 — A crow skull is a free look; place the best chip, or none.

Blue draws $k \in \{1, 2, 4\}$ extra chips, places at most one, and returns the rest. The engine enumerates every multiset of k chips that could be revealed, weighted by the hypergeometric probability

$$P(\text{reveal } m) = \Pi \text{ over types } i \text{ of } C(n_i, m_i) / C(n, k),$$

and for each reveal takes the maximum over "place chip j " and "place nothing". Placing nothing is a real option and is sometimes best — for instance when the only revealed chips are whites that would take the sum to 7, leaving no safe draws. The advisor's ranked list after a crow skull is exactly this inner maximisation for the reveal you actually got.

Maxim 7 — Beyond a short horizon, only the whites matter.

The exact recursion of Maxim 1 is affordable early in the game (a 9-chip bag has a few hundred reachable states) but not late (a 26-chip bag has millions). The engine therefore searches the full game tree for a fixed depth of draws and values the leaf with a second, exact solver on an **abstracted bag**: whites, purples and blacks kept exactly; every other chip collapsed to a "safe mover" of an effective value (red plus its pumpkin bonus, blue plus the value of choosing, yellow plus 0.6 for the bomb it may erase, green plus a fraction of a ruby). This is justified by what drives the decision: the risk is entirely in the whites, and the abstract solver treats the whites and the explosion rule exactly. On a mid-game 17-chip bag the depth-2 value agrees with the fully exact solve to within 0.12 VP-equivalents; on the starting bag they coincide. The Rust engine plays its benchmarks at depth 1, which is about a hundred times faster than the Python reference and, over 50 000 games, indistinguishable in score.

3. Maxims of shopping

Maxim 8 — Coins do not carry over, so "buy nothing" is almost never right.

Unspent coins are forfeited. The purchase problem is therefore a pure maximisation with no saving option: choose the affordable set of one or two chips of different colours that maximises the value of the resulting bag. "Nothing" is included in the ranking only so the user can see the baseline; it wins solely when the coins are below the cheapest chip (3).

Maxim 9 — Rank purchases by the value of the whole rest of the game, not by the chip's face value and not by next round alone.

The score of a candidate purchase P after round r is

$$EV(P) = V_{r+1}(\text{bag } B \cup P, \text{droplet, rubies, flask}),$$

where $V_{\{r+1\}}$ is the learned value function of Section 4. This captures what face value misses: a coloured chip *dilutes* the whites, a red chip is worth more when oranges are already in the bag, a fourth purple is worth less than the third. It also captures what a one-round look-ahead misses, which was the weakness of the v1 engine: a chip bought in round 2 brews seven more times, and an orange bought now raises the value of every red bought later. The learned policies express this as a purchase mix. Over a game the v3 policy buys, on average, 3.2 red, 2.3 orange, 2.3 purple, 2.2 blue, 1.4 black and 1.3 green chips (Appendix A); the v1 heuristic buys no red at all.

Maxim 10 — Rubies buy droplet steps and the flask when the value function says so, and are cashed for points only at the end.

Two rubies buy one of: a permanent droplet step, a flask refill, or (at game end) one VP. The engine folds the ruby decision into the shop: every purchase is evaluated jointly with every affordable ruby spend, and the combination with the highest $V_{\{r+1\}}$ is recommended. A droplet step moves *every* future starting position, so early in the game the value function prices it well above the two rubies; by round 8 the same step is worth less than holding the rubies for the final conversion. There is no separate ruby rule.

4. Maxims of value — learning what a round is worth

Maxim 11 — The value of a round's outcome is the value of the best bag it can buy.

The v1 engine converted coins, rubies and droplet steps to victory points at fixed per-round exchange rates (Table 2 of the previous edition of this paper). Those rates were the engine's only heuristic layer, and they were wrong in a specific way: they priced a coin in round 2 without knowing what the coin would buy, and they could not see that an orange and a red are worth more together than apart. The v2 and v3 engines replace them with a learned function

$V_r(\text{bag, droplet, rubies, flask, margin})$ — the value of standing at the start of round r with that state,

fitted by ridge regression on 45 features of the state (chip counts and their squares, droplet and its square, rubies, flask, total white value, number of coloured chips, orange $\times$ red count, white fraction) plus the *margin*, defined as the player's score minus the best opponent's score. One model is fitted per round from self-play, with a replay buffer across training passes so the fit does not chase a single policy. The terminal value of a brew is then

$T(s) = \max \text{ over purchases } P \text{ and ruby spends of } V_{\{r+1\}}(\text{bag after } P, \dots), \text{ with VP}(s) \text{ and the die entering through the margin (Section 5),}$

precomputed once per round as a table over (coins, rubies gained, droplet steps gained, flask used) — about 1 500 shop optimisations, 20 ms on a phone — so that the in-round solver of Section 2 remains exact and fast.

Maxim 12 — The bonus die is worth its expectation times the chance you are ahead.

The die has faces {1 VP, 1 VP, 2 VP, ruby, droplet, orange}, and each face is valued through $V_{\{r+1\}}$ (the VP faces through the margin, the ruby and droplet faces through the state, the orange through the

change in bag value). The player rolls it only if no un-exploded opponent reached a higher space. With k opponents whose best non-exploded space is modelled as $\text{Normal}(\mu_r, \sigma)$ and who each survive with probability p ,

$$P(\text{win die} \mid s) = [(1 - p) + p \cdot \Phi((s - \frac{1}{2} - \mu_r) / \sigma)]^k$$

This term is what makes the last few draws of a good round worth taking. The v3 policy rolls the die 3.2 times per game at a four-player table, the v2 policy 2.5 (Appendix B).

Maxim 13 — The value of a ruby space is one ruby, nothing more; the value of a green chip is a ruby *sometimes*.

Rubies on the track are paid whether or not the pot exploded, so they enter T unconditionally. A green chip pays a ruby only if it is one of the last two chips placed; the engine tracks the colours of the last two chips in the state and counts greens there at the terminal. The learned policy, which prices rubies through what they buy, values green well below the v1 heuristic did: it buys 1.3 green chips per game against v1's 2.9.

5. Maxims of winning

Maxim 14 — The objective is first place; expected score is only a proxy, and a proxy that fails exactly when the game is close.

Maximising expected final score treats a +3 and a −3 swing as cancelling, and never looks at the other players. Maximising win probability is a different objective in three ways. It is **relative**: points that every seat receives are worthless, and rat tails become a cost when leading, since they feed the players behind. It is **positional**: the same bag is worth a different action ten points ahead than ten points behind. And it is **risk-aware**: a trailing player should seek variance and a leading player should shed it, because a safe stop that still loses is worth nothing. The v2 engine could express none of these. The v3 engine expresses all three through one number, the margin, and one function of it.

Maxim 15 — Value the bag in points, then map the projected final margin to a win probability.

Fitting a win probability directly to the state — a logistic regression of the 0/1 win indicator on the 46 features — was tried first and performs badly: the win indicator is a far noisier signal than the score, and a linear-logit model forces a convex (gambling) shape onto every seat that is below average, which at a four-player table is three seats in four. The policy it produced exploded in half its rounds and won 12 % of games against v2.

The formulation that works separates the two questions. First, the bag is valued in points, from the low-noise score signal, by the ridge regression of Maxim 11; at a real table this regression includes the margin, and finds that each point of lead costs between 0.6 (round 2) and 0.1 (round 9) points of future income, which is the rat-tail mechanism measured. Second, the *projected final margin*

$$x_r = \text{margin} + V_r(\text{state}) - G_r$$

where G_r is the mean score the other players still gain from round r on, is mapped to a win probability by a two-parameter logistic fitted per round:

$$P(\text{win} \mid \text{state at round } r) = \sigma(c_r + a_r \cdot x_r), \sigma(z) = 1 / (1 + e^{-z}).$$

Because the composition is linear in the state features up to the sigmoid, it is stored and evaluated exactly as the v2 model was — one coefficient vector per round, plus a tenth for the end of the game — and the shop ranking of Maxim 9 is unchanged (the sigmoid is monotone, so the best purchase does not depend on the margin). Table 2 gives the fitted parameters. The slope a_r rises from 0.18 per point in round 2 to 0.60 at the end of the game: early on, a point is nearly weightless in win terms and the policy plays for the bag; late, every point is decisive.

Table 2. The v3 win model by round: rat-tail coefficient of the score regression (points of future income lost per point of current lead), mean opponent gain in the round, and the logistic map from projected final margin to win probability. Fitted on 20 000 seat-games.

round	1	2	3	4	5	6	7	8	9	end
lead cost b_r	0	-0.63	-0.56	-0.51	-0.46	-0.40	-0.33	-0.24	-0.13	—
opponent gain (VP)	1.0	1.3	1.9	2.8	4.0	5.0	6.0	7.1	12.4	—
intercept c_r	-1.00	-0.82	-0.75	-0.71	-0.64	-0.58	-0.54	-0.44	-0.27	-0.23
slope a_r (per VP)	0	0.18	0.22	0.23	0.24	0.26	0.29	0.33	0.41	0.60

Maxim 16 — Risk appetite is not a setting; it is the curvature of the sigmoid at your margin.

During a brew the terminal value of an outcome that gains g points and buys the best bag with its coins is

$$T = \sigma(z(\text{bag after shop}) + a_{\{r+1\}} \cdot (\text{margin} - \text{opp_gain}_r + g)),$$

where z is the tabled logit of Maxim 11 and opp_gain_r is the mean score the opponents gain this round. Nothing else in the solver changes. But σ is convex below its midpoint and concave above it, so the expectation over a risky draw is worth *more* than the sure stop when the player is behind and *less* when ahead — by exactly the amount the fitted slope says a point is worth at that stage of the game. This is the risk flip of Maxim 14, and it costs no rule: in round 9, trailing by four, the v3 engine keeps drawing at explosion odds the v2 engine would refuse, because the sure stop has a win probability near zero; leading by four, it stops earlier than v2 would, because the sure stop already wins.

The same formula settles the VP-or-coins choice after an explosion: keep the points if $a \cdot \text{VP}(s)$ exceeds the gain in z that the coins would buy. Written in points, that is the v2 rule; written in logits, it is the v3 rule, and the scale factor a between them is not optional. An early version of the engine compared points to logits directly, which made one point outweigh any purchase and hid the merit of the whole approach until it was found (Appendix C).

Maxim 17 — Train against the best you have, and keep a policy only if it wins more than it loses.

Self-play against copies of oneself cannot rank policies at a table: rat tails redistribute score, so the mean score of four clones is nearly the same whatever they play (41.4 ± 0.1 VP for every policy tried).

The v3 trainer therefore seats each new candidate in two chairs against the best policy so far in the other two, and adopts it only if the candidate's chairs win more than half the games. The value function is refitted after every pass from all seats of all passes. The adopted v3 policy won 56 % of such games against v2 in training and, evaluated without exploration, 34 % of four-player games against three copies of v2 (Appendix B).

Maxim 18 — Opponents are weather within a round, and players between rounds.

The solver never models what an opponent *chooses* during a brew: it models what an opponent's position *does to you*. At the table the between-round channels are read from the real scores — the advisor asks for the best opponent's score each round, from which it computes both the rat tails and the margin — while the within-round channels (who rolls the die, who wins the black-chip comparison) are unknown until every pot has stopped and are priced by the calibrated curves of Maxim 12 and Maxim 19. In the simulator the same split holds: the table mode settles rat tails, the die and the black chips from the seats' actual results, and each seat's solver prices them as the advisor would.

Maxim 19 — Black chips are a bet on the neighbours, priced as one.

A black chip (moth) pays a droplet step if you have more blacks than either neighbour and a ruby as well if you beat both. With each neighbour's count modelled as $\text{Poisson}(\mu_r)$,

$$P(\text{droplet}) = 1 - (1 - P(X < b))^2, P(\text{ruby}) = P(X < b)^2$$

for b blacks in your pot (two-player games compare against a single opponent, with ties also paying the droplet). Early in the game μ is near zero and a single black is a near-certain droplet step every round — which is why every learned policy buys a black in round 1 and why the v3 policy, which values the bag more aggressively, buys 1.4 per game.

6. What to verify at the table

The engine is exact about the rules it was given; its accuracy at your table depends on inputs that were reconstructed rather than read from the box.

1. **The pot track** (Table 1) was transcribed from a photograph of the board and is believed correct, including the second "33" carrying a ruby.
2. **Rat tails** on the VP scoring track are assumed at VP 1, 3, 6, 8, 10, 12, ... 50 (every second space after 6). Under the win objective this matters more than before, since rat tails are the mechanism behind the lead-cost coefficients of Table 2.
3. **The bonus die's sixth face** is assumed to be a second "1 VP".
4. **Fortune-teller cards** are not modelled in the simulator. In the advisor they are a small set of switches (explosion limit 8 or 9, droplet +1 for the round, exploded pots keep both, +1/+2 VP or +1 ruby for not exploding). Cards outside that set are not modelled; when one is drawn, enter nothing and treat the advice as slightly conservative. Most cards apply to every player and therefore cancel under the win objective.
5. **The die and black-chip curves** of Maxims 12 and 19 were calibrated against the engine's own play. A table that explodes more often than the simulated one hands out the die more freely; the draw/stop decisions would move a little towards drawing.

6. **The opponent-gain curve** of Table 2 is the mean of the simulated table. Against markedly stronger or weaker human opponents the projected margin is biased by the difference, which shifts the risk appetite of Maxim 16 but not the ranking of purchases.

7. Summary of the maxims

1. Draw when $E[\text{draw}] > \text{value of stopping}$; nothing else is a rule.
2. A draw that cannot explode is always taken.
3. An explosion is a purchase; early ones are cheap.
4. The flask erases bombs; use it when the position saved beats the refill.
5. A mandrake almost always returns the white before it.
6. A crow skull is a free look; place the best chip, or none.
7. Beyond a short horizon, only the whites matter.
8. Coins don't carry over, so buy something.
9. Rank purchases by the value of the rest of the game.
10. Rubies buy droplet steps and the flask when the value function says so.
11. A round is worth the best bag it can buy.
12. The die is worth its expectation times the chance you're ahead.
13. A ruby space is one ruby; a green chip is a ruby *sometimes*.
14. The objective is first place; expected score is a proxy that fails when it's close.
15. Value the bag in points, then map the projected margin to a win probability.
16. Risk appetite is the curvature of the sigmoid at your margin.
17. Train against the best you have; keep a policy only if it wins more than it loses.
18. Opponents are weather within a round, and players between rounds.
19. Black chips are a bet on the neighbours, priced as one.

Appendix A — Solo benchmarks

Solo play against the modelled opponent curves (no real table), Rust engine, depth 1. Means over 50 000 games for v1 and v2, 3 000 for v3; the solo score is *not* the objective of v3 and is reported for continuity.

policy	mean VP	sd	explode % (rounds 1–9)	bust → coins % (rounds 1–9)	chips bought per game O / G / B / R / Y / P / K
v1 heuristic	37.8	6.0	34, 33, 46, 29, 23, 17, 14, 14, 17	100, 100, 100, 100, 100, 100, 87, 16, 1	0.9 / 3.0 / 2.5 / 0.0 / 0.4 / 0.6 / 1.8
v2 expected score	41.2	6.5	53, 34, 40, 24, 20, 24, 35, 58, 21	100, 100, 100, 100, 66, 3, 0, 0, 0	1.6 / 2.6 / 2.3 / 0.6 / 0.0 / 2.9 / 1.0
v3 win probability	40.1	6.8	62, 54, 56, 31, 24, 21, 27, 47, 41	100, 100, 100, 100, 100, 91, 2, 0, 0	2.3 / 1.3 / 2.2 / 3.2 / 0.0 / 2.3 / 1.4

The v3 policy scores a point less than v2 when played alone, because the solo opponent curve gives it no real leader to react to; its busts in rounds 1–3 and round 9 are bets that pay off only at a table.

Appendix B — Four-player table

The simulator's table mode seats two to four policies, each brewing with its own solver, and settles rat tails, the bonus die and the black-chip comparison from the seats' actual results. 4 000 games per table; a win is a share of first place (ties split). Standard error of a win rate is about 0.7 points.

B.1 v3 against three copies of v2.

seat	policy	mean VP	win %	rat tails / game	die rolls / game	explode %
1	v3	43.5	34.3	12.1	3.24	41.7
2	v2	42.8	24.1	8.6	2.44	34.3
3	v2	43.1	25.8	8.1	2.52	33.9
4	v2	42.7	23.9	8.5	2.45	34.3

B.2 Mixed tables. Two fields, 4 000 games each. Seat order itself has no measurable effect (rotating a fixed field reproduces every seat's numbers to within a few tenths); the field does, because wins are shared among four.

policy	field A win %	field B win %
v3	31.8	37.6
v2	24.0	29.1
v1	18.6	22.1
v2, shop restricted to orange/red/blue/black	—	19.5
v3, earlier fit	33.1	—

The v3 policy's edge over v2 is 8–10 points of win rate in every field, on a mean-score edge of under one point. That is the content of Maxim 14: a small gain in score, spent in the right rounds, becomes a large gain in first places.

Appendix C — Method notes

Engines. The Python engine (`brew.py`, `shop.py`, `value.py`) is the v1 reference and agrees with the Rust engine (`rust/`) to four decimals on the parity bags. The Rust engine adds the learned value functions, the self-play trainer (`train`), the table mode (`table`) and the win-objective trainer (`train --table`). The JavaScript in `app/index.html` is a line-for-line port of the solver and value model and agrees with the Rust engine to four decimals on the same bags with the v3 weights installed.

Fits. Ridge regression ($\lambda = 10$) per round on the score still to come; the 2-parameter logistic per round by damped Newton iteration on the win indicator, with the projected final margin of Maxim 15 as its single feature. Each training pass plays 5 000 four-player tables (20 000 seat-games) with 20 % random purchases for exploration, then refits from the replay buffer of every pass so far.

A note on the bug that hid the result. The first three training runs of the win objective produced policies that lost 75–90 % of games to v2. The cause was a single line in the post-explosion settlement — the choice to keep points or coins — that compared points to logits without the scale factor a of Maxim 16, so every win-objective policy kept the points after every bust. With the line corrected, the same value

functions won. The lesson is recorded here because it is general: when a value function changes units, every consumer of it must change with it, and the ones outside the solver are the easiest to miss.

Reproducing the numbers. From `rust/: quackulator sim --games 50000 (v1 solo), sim --weights weights.json (v2 solo), sim --weights weights_win.json (v3 solo); table --players weights_win.json,weights.json,weights.json,weights.json --games 4000 for B.1; train --table --games 5000 --passes 4 --explore 0.2 --init weights.json --out weights_win.json` retrains v3 in about ten minutes on a desktop CPU.

Contributors

Corey D. — design, rules model, engine, experiments, and this paper. Claude (Anthropic) — contributor: Rust table mode and win-objective trainer, app port of the win model, and drafting assistance on this edition.